Registers

General Purpose Registers

Register	ABI Name	Recommended Usage (ABI)
`x0`	`zero`	Hard-wired to zero
`x1`	`ra`	Return Address
`x2`	`sp`	Stack pointer
`x3`	`gp`	Global pointer
`x4`	`tp`	Thread pointer
`x5` - `x7`	`t0-t2`	Temporary registers
`x8`	`s0 / fp`	Frame pointer
`x9`	`s1`	Saved register
`x10` - `x17`	`a0-a7`	Function argument / Return value
`x18` - `x27`	`s2-s11`	Saved registers
`x28` - `x31`	`t3-t6`	Temporary registers

Registers `ra`, `t0` - `t7`, and `a0` - `a7` may be freely overwritten by any function. Hence, they are caller saved. All other registers must be saved by the callee, if overwritten.

Control and Status Registers

Register	Usage
`mstatus`	Machine status (global interrupt enables, privilege bits)
`mcause`	Cause of last exception (MSB is `1` if interrupt, `0` otherwise)
`mepc`	Exception program counter (return address)
`mtval`	Trap value (faulting address or instruction)
`mtvec`	Machine exception vector base address
`mie`	Machine interrupt enable

Instructions

Abbreviations

rd : Destination register
rs, rs1, rs2 : Source registers
imm: Immediate value.
sext: Sign Extend
zext: Zero Extend

Immediate & Address Construction

Instruction	Mnemonic	Result
`lui rd, imm`	Load upper immediate	rd ← imm << 12
`auipc rd, imm`	Add upper immediate to PC	rd ← PC + (imm << 12)
`li rd, imm`	Load immediate	rd ← imm
`la rd, symbol`	Load address	rd ← address
`lla rd, symbol`	Load local address	rd ← local address

Shifts & Bit Manipulation

Instruction	Mnemonic	Result
`slli rd, rs1, imm`	Shift left logical immediate	rd ← rs1 << imm
`srli rd, rs1, imm`	Shift right logical immediate	rd ← zext(rs1 >> imm)
`srai rd, rs1, imm`	Shift right arithmetic immediate	rd ← sext(rs1 >> imm)
`not rd, rs`	Bitwise NOT	rd ← ~rs

Integer Arithmetic & Logic

Instruction	Mnemonic	Result
`addi rd, rs1, imm`	Add immediate	rd ← rs1 + imm
`xori rd, rs1, imm`	XOR immediate	rd ← rs1 ^ imm
`ori rd, rs1, imm`	OR immediate	rd ← rs1 | imm
`andi rd, rs1, imm`	AND immediate	rd ← rs1 & imm
`add rd, rs1, rs2`	Add	rd ← rs1 + rs2
`sub rd, rs1, rs2`	Subtract	rd ← rs1 − rs2
`xor rd, rs1, rs2`	XOR	rd ← rs1 ^ rs2
`or rd, rs1, rs2`	OR	rd ← rs1 | rs2
`and rd, rs1, rs2`	AND	rd ← rs1 & rs2
`mul rd, rs1, rs2`	Multiply (lower 32 bits)	rd ← (rs1 × rs2)[31:0]
`div rd, rs1, rs2`	Divide (signed)	rd ← rs1 ÷ rs2 (signed)
`divu rd, rs1, rs2`	Divide (unsigned)	rd ← rs1 ÷ rs2 (unsigned)
`rem rd, rs1, rs2`	Remainder (signed)	rd ← rs1 % rs2 (signed)
`remu rd, rs1, rs2`	Remainder (unsigned)	rd ← rs1 % rs2 (unsigned)

Load and Store Instructions

Instruction	Mnemonic	Result
`lb rd, offset(rs1)`	Load byte signed	rd ← sext(M(rs1 + offset)) [1 byte]
`lh rd, offset(rs1)`	Load halfword signed	rd ← sext(M(rs1 + offset)) [2 bytes]
`lw rd, offset(rs1)`	Load word	rd ← M(rs1 + offset) [4 bytes]
`lbu rd, offset(rs1)`	Load byte unsigned	rd ← zext(M(rs1 + offset)) [1 byte]
`lhu rd, offset(rs1)`	Load halfword unsigned	rd ← zext(M(rs1 + offset)) [2 bytes]
`sb rs2, offset(rs1)`	Store byte	M(rs1 + offset) ← rs2 [1 byte]
`sh rs2, offset(rs1)`	Store halfword	M(rs1 + offset) ← rs2 [2 bytes]
`sw rs2, offset(rs1)`	Store word	M(rs1 + offset) ← rs2 [4 bytes]
`lb rd, label`	Load byte signed	rd ← sext(M(label)) [1 byte]
`lbu rd, label`	Load byte unsigned	rd ← zext(M(label)) [1 byte]
`lh rd, label`	Load halfword signed	rd ← sext(M(label)) [2 bytes]
`lhu rd, label`	Load halfword unsigned	rd ← zext(M(label)) [2 bytes]
`lw rd, label`	Load word	rd ← M(label) [4 bytes]

Control Flow Instructions

Instruction	Mnemonic	Result
`jal rd, offset`	Jump and link	rd ← PC+4; PC ← PC+offset
`jalr rd, offset(rs1)`	Jump and link register	rd ← PC+4; PC ← rs1+offset
`beq rs1, rs2, offset`	Branch if equal	PC ← PC+offset if rs1 = rs2
`bne rs1, rs2, offset`	Branch if not equal	PC ← PC+offset if rs1 ≠ rs2
`blt rs1, rs2, offset`	Branch if less than (signed)	PC ← PC+offset if rs1 < rs2
`bge rs1, rs2, offset`	Branch if greater or equal (signed)	PC ← PC+offset if rs1 ≥ rs2
`bltu rs1, rs2, offset`	Branch if less than (unsigned)	PC ← PC+offset if rs1 < rs2 (u)
`bgeu rs1, rs2, offset`	Branch if greater or equal (unsigned)	PC ← PC+offset if rs1 ≥ rs2 (u)
`beqz rs, offset`	Branch if equal to zero	PC ← PC+offset if rs = 0
`bnez rs, offset`	Branch if not equal to zero	PC ← PC+offset if rs ≠ 0
`bgtz rs, offset`	Branch if greater than zero	PC ← PC+offset if rs > 0
`bltz rs, offset`	Branch if less than zero	PC ← PC+offset if rs < 0
`bgt rs1, rs2, offset`	Branch if greater than (signed)	PC ← PC+offset if rs1 > rs2
`ble rs1, rs2, offset`	Branch if less or equal (signed)	PC ← PC+offset if rs1 ≤ rs2
`bgtu rs1, rs2, offset`	Branch if greater than (unsigned)	PC ← PC+offset if rs1 > rs2 (u)
`bleu rs1, rs2, offset`	Branch if less or equal (unsigned)	PC ← PC+offset if rs1 ≤ rs2 (u)
`tail rs`	Tail call	PC ← rs (no return)
`j label`	Unconditional jump	PC ← label
`jr rs`	Jump register	PC ← rs
`ret`	Return	PC ← ra
`call symbol`	Call function	ra ← PC+4; PC ← fn

CSR Instructions

Instruction	Mnemonic	Result
`csrr rd, csr`	Read CSR	rd ← csr
`csrw csr, rs`	Write CSR	csr ← rs
`csrs csr, rs`	Set CSR bits	csr ← csr | rs
`csrc csr, rs`	Clear CSR bits	csr ← csr & ~rs

Common RISC-V Assembler Directives (GAS)

This chapter summarises the most important assembler directives used in RISC-V assembly programming with GNU assembly.

Note:

- Directives are assembler instructions, not CPU instructions.
- They do not generate executable code directly.
- Exact behaviour can vary slightly between toolchains.

Data definition directives

Define raw data in memory.
Most data directives accept one or more comma-separated values.

- `.byte expr [, expr ...]`
  Emit one or more 8-bit values.

```
my_byte:     .byte 0x12
my_bytes:    .byte 1, 2, 3, 4
```

- `.half expr [, expr ...]`
  Emit one or more 16-bit values.

```
my_half:     .half 0x1234
my_halves:   .half 10, 20, 30
```

- `.word expr [, expr ...]`
  Emit one or more 32-bit values (RV32).

```
my_word:     .word 0x12345678
my_words:    .word 100, 200, 300
```

- `.dword expr [, expr ...]`
  Emit one or more 64-bit values (RV64).

```
my_dword:    .dword 0x1122334455667788
```

- `.asciz "string"`
  Emit a null-terminated string.

```
message:     .asciz "Hello, world"
```

- `.ascii "string"`
  Emit a string without a terminating null byte.

```
banner:      .ascii "RISC-V"
```

Space and alignment

Reserve or align storage.

- `.space n`
  Reserve n bytes (contents unspecified or zeroed by linker).

```
buffer:      .space 64
```

- `.align n`
  Align the location counter to a 2^n-byte boundary.

```
.align 2     # align to 4-byte boundary
```

- `.balign n`
  Align to an n-byte boundary.

```
.balign 16
```

Symbol visibility and linkage

Control symbol scope and linkage.

- `.global symbol` (or `.globl`)
  Make symbol visible to the linker.

```
.global main
```

- `.extern symbol`
  Declare a symbol defined in another file.

```
.extern printf
```

- `.local symbol`
  Make a symbol local to this object file.

Sections

Select where code and data are placed.

- `.text`
  Code section.
- `.data`
  Writable data section.
- `.rodata`
  Read-only data.
- `.bss`
  Uninitialised zero-filled data.

Labels and constants

- `label:`
  Define a symbol at the current address.
- `.equ name, value`
  Define a symbolic constant.

```
.equ BUF_SIZE, 64
```

- `.set name, value`
  Alternative to .equ.

Function-related directives (conventions)

- `.type symbol, @function`
  Mark a symbol as a function.
- `.size symbol, .-symbol`
  Define the size of a symbol.

GPIO

The General Purpose Input Output module consists of five identical register blocks, one for each GPIO port.

Base addresses

GPIOA: `0x40010800`
GPIOB: `0x40010C00`
GPIOC: `0x40011000`
GPIOD: `0x40011400`
GPIOE: `0x40011800`

Register Block Overview

offset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Register
0x0																																	CFGLR
0x4																																	CFGHR
0x8																																	INDR
0xC																																	OUTDR
0x10																																	BSHR
0x14																																	BCR
0x18																																	LCKR

CFGLR

Configuration of GPIO port, for lower 8 pins. Configuration of the port is done using the CFGLR and CFGHR for the lower eight and upper eight pins, respectively. For each pin, there are two bits to set the MODE (Input or Output, and speed of output pins), and two bits to set the CoNFiguration (the meaning of these pins depend on whether the pin is an input pin or an output pin):

CNF		MODE	
00	Analog	00	Input Mode
01	Floating		
10	Pull-up/Pull-down		
11	Reserved		
00	Push Pull	01 10 11	Output Mode 10Mhz Output Mode 2Mhz Output Mode 50Mhz
01	Open Drain		
10	Alternate Function, Push Pull		
11	Alternate Function, Open Drain		

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF7		MODE7		CNF6		MODE6		CNF5		MODE5		CNF4		MODE4		CNF3		MODE3		CNF2		MODE2		CNF1		MODE1		CNF0		MODE0	

CNFn (n = 0..7): Set CONF for pin n
MODEn (n = 0..7): Set MODE for pin n

CFGHR

Configure port for the upper 8 pins. (See CFGLR for details)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF15		MODE15		CNF14		MODE14		CNF13		MODE13		CNF12		MODE12		CNF11		MODE11		CNF10		MODE10		CNF9		MODE9		CNF8		MODE8	

CNFn (n = 8..15): Set CONF for pin n
MODEn (n = 8..15): Set MODE for pin n

INDR

Port input data register. 16 bits register, one bit per pin.
IDRn (n = 0..15): Reflects the logic level of pin n.

OUTDR

Port output data register. 16 bits register, one bit per pin.
ODRn (n = 0..15):
If pin is configured as **output**: Set logic value of GPIO Port pin n.
If pin is configured as **input**: Configures pull-up (1) or pull-down (0), if enabled in CFGLR/CFGHR.
Reading is allowed, but reflects the status of the OUTDR register, not the pin.

BSHR

Port bit set/reset register (GPIOn_BSHR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0	BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0

BRn (n = 0..15): Writing 1 to BRn resets bit n in OUTDR to 0
BSn (n = 0..15): Writing 1 to BSn sets bit n in OUTDR to 1

BCR

Port bit reset register. 16 bits register, one bit per pin.
BRn (n = 0..15): Writing 1 to BRn resets bit n in OUTDR to 0

SysTick

Base address:

`0xE000F000`

Register Block Overview

offset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Register
0x0																																	STK_CTLR
0x4																																	STK_SR
0x8																																	STK_CNTL
0xC																																	STK_CNTH
0x10																																	STK_CMPLR
0x14																																	STK_CMPHR

STK_CTLR

System counter control register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIE																										INIT	MODE	STRE	STCLK	STIE	STE

SWIE: If set, immediately triggers a SysTick Interrupt.
INIT: When this bit is set, timer loads CNT with 0 if upcounting or with CMP if downcounting. Writing zero is ignored.
MODE: If set, count down, otherwise count up.
STRE: If set, restarts from 0 when CMP reached (upcounting) or from CMP when zero is reached (downcounting). Otherwise it keeps counting beyond the compare value (or zero).
STCLK: Selects the clock source: the processor clock (HCLK) when set, or HCLK/8 if not set.
STIE: Enables SysTick interrupt generation when the counter reaches zero; if cleared, no interrupt is triggered.
STE: Turns the SysTick timer on when set; clearing it stops the counter.

STK_SR

Status Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
																															CNTIF

CNTIF: Is set to 1 when CNT reaches CMP (if upcounting) or when CNT reaches 0 (if downcounting). Must be cleared by software, or the interrupt will immediately re-trigger. Reset by writing 0.

STK_CNTL

System counter low register.
CNTL: Lower 32-bit of the system counter value (CNT).

STK_CNTH

System counter high register.
CNTH: Upper 32-bit of the system counter value (CNT).

STK_CMPLR

System compare low register.
CMPL: Lower 32-bit of the compare value (CMP).

STK_CMPHR

System compare high register.
CMPH: Upper 32-bit of the compare value (CMP).

AFIO

The Alternate Function I/O (AFIO) module controls pin remapping and event/interrupt configuration. It allows GPIO pins to be reassigned from their default peripheral functions to alternate ones, and configures which GPIO port supplies each EXTI interrupt line.

Base addresses

`0x40010000`

Register Block Overview

offset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Register
0x0																																	ECR
0x4																																	PCFR
0x08 - 0x14	*Group: External interrupt configuration register 1 (AFIO_EXTICR1)*																																EXTICR1 .. EXTICR4
0x1C																																	PCFR2

EXTICR1 ... EXTICR4

External interrupt configuration register 1 (AFIO_EXTICR1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
																EXTI3				EXTI2				EXTI1				EXTI0			

EXTIn (n = 0..3): Port selection for pin n. 0: Port A, 1: Port B, ... , 4: Port E

External interrupt configuration register 2 (AFIO_EXTICR2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
																EXTI7				EXTI6				EXTI5				EXTI4			

EXTIn (n = 4..7): Port selection for pin n. 0: Port A, 1: Port B, ... , 4: Port E

External interrupt configuration register 3 (AFIO_EXTICR3)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
																EXTI11				EXTI10				EXTI9				EXTI8			

EXTIn (n = 8..11): Port selection for pin n. 0: Port A, 1: Port B, ... , 4: Port E

External interrupt configuration register 4 (AFIO_EXTICR4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
																EXTI15				EXTI14				EXTI13				EXTI12			

EXTIn (n = 12..15): Port selection for pin n. 0: Port A, 1: Port B, ... , 4: Port E

EXTI

The EXTI module decides whether an interrupt should be generated and sent onto the PFIC. Each register listed in the block overview below contains 20 bits, each corresponding to one of the EXTI lines 0-19.

Base addresses

`0x40010400`

Register Block Overview

offset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Register
0x00																																	INTENR
0x04																																	EVENR
0x08																																	RTENR
0xC																																	FTENR
0x10																																	SWIEVR
0x14																																	INTFR

INTENR

Interrupt mask register (EXTI_INTENR)
MRn (n = 0..19): Interrupt Mask on line n. Indicates whether interrupt is enabled.

RTENR

Rising Trigger selection register (EXTI_RTENR)
TRn (n = 0..19): Rising edge trigger event configuration of line n.

FTENR

Falling Trigger selection register (EXTI_FTENR)
TRn (n = 0..19): Falling edge trigger event configuration of line n.

INTFR

Interrupt Flag Register (pending) (EXTI_INTFR)
PRn (n = 0..19): Pending bit (write 1 to clear)

PFIC

The PFIC (Programmable Fast Interrupt Controller) is responsible for managing interrupts and events for the microcontroller. It provides a centralized interface for handling various peripheral interrupts.

Base addresses

`0xE000E000`

Register Block Overview

offset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Register
0x00 - 0x0C	*Group: Interrupt Status Register*																																ISR1 .. ISR4
0x20 - 0x2C	*Group: Interrupt Pending Register*																																IPR1 .. IPR4
0x40																																	ITHRESDR
0x48																																	CFGR
0x4C																																	GISR
0x50																																	VTFIDR
0x60 - 0x6C	*Group: Interrupt 0 address Register*																																VTFADDRR0 .. VTFADDRR3
0x100 - 0x10C	*Group: Interrupt Enable Set Register*																																IENR1 .. IENR4
0x180 - 0x18C	*Group: Interrupt Clear Register*																																IRER1 .. IRER4
0x200 - 0x20C	*Group: Interrupt Pending Register*																																IPSR1 .. IPSR4
0x280 - 0x28C	*Group: Interrupt Pending Clear Register*																																IPRR1 .. IPRR4
0x300 - 0x30C	*Group: Interrupt ACTIVE Register*																																IACTR1 .. IACTR4
0x400 - 0x4FF	*Group: Interrupt Priority Register*																																IPRIOR0 .. IPRIOR255
0xd10																																	SCTLR

ISR1 ... ISR4

Interrupt Status Registers

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INTENSTA12_31																												INTENSTA2_3			

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INTENSTA																															

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INTENSTA																															

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
																								INTENSTA							

INTENSTA: Interrupt status. One bit per interrupt. Bit number x in ISRn corresponds to interrupt 32 * n + x. Stores 1 if interrupt is enabled, 0 if interrupt is disabled. Read-only registers.

IPR1 ... IPR4

Interrupt Pending Registers

Stores pending status of interrupts. Bit number x in IPRn corresponds to interrupt 32 * n + x. Stores 1 if interrupt is pending, 0 if interrupt is not pending. Read-only registers.

IENR1 ... IENR4

Interrupt Enable Set Registers

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INTEN																															

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INTEN																															

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INTEN																															

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
																								INTEN							

INTEN: Bit number x in IENRn corresponds to interrupt 32 * n + x. Writing 1 enables interrupt. Write-only registers.

IRER1...IRER4

Interrupt Clear Register

Write-only registers. One bit per interrupt. Writing 1 disables interrupt. Bit number x in IRERn corresponds to interrupt 32 * n + x.

CH32V307 Interrupt Vector Table

No.	Priority	Name	Description	Address
0	-	-	Reserved	0x00000000
1	-	-	Reserved	0x00000004
2	-5 (fixed)	NMI	Non-maskable interrupt	0x00000008
3	-4 (fixed)	HardFault	Hard fault exception	0x0000000C
4	-	-	Reserved	0x00000010
5	-3 (fixed)	ECALL-M	Machine-mode ECALL	0x00000014
6–7	-	-	Reserved	0x00000018–1C
8	-2 (fixed)	ECALL-U	User-mode ECALL	0x00000020
9	-1 (fixed)	BreakPoint	Breakpoint exception	0x00000024
10–11	-	-	Reserved	
12	0	SysTick	System timer interrupt	0x00000030
13	-	-	Reserved	0x00000034
14	1	SW	Software interrupt	0x00000038
15	-	-	Reserved	0x0000003C

Peripheral Interrupts (Programmable: 16–103)

No.	Name	Description	Address
16	WWDG	Window watchdog	0x00000040
17	PVD	PVD via EXTI	0x00000044
18	TAMPER	Tamper interrupt	0x00000048
19	RTC	RTC global interrupt	0x0000004C
20	FLASH	Flash global interrupt	0x00000050
21	RCC	RCC global interrupt	0x00000054
22	EXTI0	EXTI line0 interrupt	0x00000058
23	EXTI1	EXTI line1 interrupt	0x0000005C
24	EXTI2	EXTI line2 interrupt	0x00000060
25	EXTI3	EXTI line3 interrupt	0x00000064
26	EXTI4	EXTI line4 interrupt	0x00000068
27	DMA1_CH1	DMA1 Channel 1	0x0000006C
28	DMA1_CH2	DMA1 Channel 2	0x00000070
29	DMA1_CH3	DMA1 Channel 3	0x00000074
30	DMA1_CH4	DMA1 Channel 4	0x00000078
31	DMA1_CH5	DMA1 Channel 5	0x0000007C
32	DMA1_CH6	DMA1 Channel 6	0x00000080
33	DMA1_CH7	DMA1 Channel 7	0x00000084
34	ADC1_2	ADC1/ADC2 global	0x00000088
35	USB_HP/CAN1_TX	USB FS HP or CAN1 TX	0x0000008C
36	USB_LP/CAN1_RX0	USB FS LP or CAN1 RX0	0x00000090
37	CAN1_RX1	CAN1 RX1	0x00000094
38	CAN1_SCE	CAN1 status/error	0x00000098
39	EXTI9_5	EXTI lines 5–9	0x0000009C
40	TIM1_BRK	TIM1 break	0x000000A0
41	TIM1_UP	TIM1 update	0x000000A4
42	TIM1_TRG_COM	TIM1 trigger/comm	0x000000A8
43	TIM1_CC	TIM1 capture/compare	0x000000AC
44	TIM2	TIM2 global	0x000000B0
45	TIM3	TIM3 global	0x000000B4
46	TIM4	TIM4 global	0x000000B8
47	I2C1_EV	I2C1 event	0x000000BC
48	I2C1_ER	I2C1 error	0x000000C0
49	I2C2_EV	I2C2 event	0x000000C4
50	I2C2_ER	I2C2 error	0x000000C8
51	SPI1	SPI1 global	0x000000CC
52	SPI2	SPI2 global	0x000000D0
53	USART1	USART1 global	0x000000D4
54	USART2	USART2 global	0x000000D8
55	USART3	USART3 global	0x000000DC
56	EXTI15_10	EXTI lines 10–15	0x000000E0
57	RTCAlarm	RTC alarm via EXTI	0x000000E4
58	USBWakeUp	USB wakeup via EXTI	0x000000E8
59	TIM8_BRK	TIM8 break	0x000000EC
60	TIM8_UP	TIM8 update	0x000000F0
61	TIM8_TRG_COM	TIM8 trigger/comm	0x000000F4
62	TIM8_CC	TIM8 capture/compare	0x000000F8
63	RNG	Random number generator	0x000000FC
64	Reserved	—	0x00000100
65	SDIO	SDIO global	0x00000104
66	TIM5	TIM5 global	0x00000108
67	SPI3	SPI3 global	0x0000010C
68	USART4	USART4 global	0x00000110
69	USART5	USART5 global	0x00000114
70	TIM6	TIM6 global	0x00000118
71	TIM7	TIM7 global	0x0000011C
72	DMA2_CH1	DMA2 Channel 1	0x00000120
73	DMA2_CH2	DMA2 Channel 2	0x00000124
74	DMA2_CH3	DMA2 Channel 3	0x00000128
75	DMA2_CH4	DMA2 Channel 4	0x0000012C
76	DMA2_CH5	DMA2 Channel 5	0x00000130
77	ETH	Ethernet global interrupt	0x00000134
78	ETH_WKUP	Ethernet wakeup interrupt	0x00000138
79	CAN2_TX	CAN2 TX	0x0000013C
80	CAN2_RX0	CAN2 RX FIFO 0	0x00000140
81	CAN2_RX1	CAN2 RX FIFO 1	0x00000144
82	CAN2_SCE	CAN2 status/error	0x00000148
83	OTG_FS	USB FS OTG interrupt	0x0000014C
84	USBHSWakeUp	USBHS wakeup	0x00000150
85	USBHS	USBHS global	0x00000154
86	DVP	Digital video port	0x00000158
87	USART6	USART6 global	0x0000015C
88	USART7	USART7 global	0x00000160
89	USART8	USART8 global	0x00000164
90	TIM9_BRK	TIM9 break	0x00000168
91	TIM9_UP	TIM9 update	0x0000016C
92	TIM9_TRG_COM	TIM9 trigger/comm	0x00000170
93	TIM9_CC	TIM9 capture/compare	0x00000174